

Hito 6 del TPO: Patrones Creacionales I

Singleton y Factory Method en LogiSmart

Duración: 90 minutos de práctica

Objetivo: Aplicar Singleton y Factory Method al diseño de LogiSmart

Entregable: Documento Markdown + Código Java + Diagramas

1. Contexto

En los hitos anteriores, creaste un diseño robusto de LogiSmart usando GRASP y patrones de asignación de responsabilidades. Ahora, en el Hito 6, vas a aplicar los **Patrones Creacionales** para mejorar cómo se crean los objetos en tu sistema.

Los dos patrones que aplicarás son:

- 1 **Singleton:** Para gestionar recursos únicos (base de datos, logger, configuración)
 - 2 **Factory Method:** Para crear diferentes tipos de objetos sin acoplamiento
-

2. Actividades (90 minutos)

Actividad 1: Identificar Candidatos a Singleton (15 minutos)

Revisa tu diseño actual y identifica qué recursos deben ser únicos en toda la aplicación.

Preguntas a responder:

- 3 ¿Qué necesita ser una única instancia en LogiSmart?
- 4 ¿Por qué debe ser única?
- 5 ¿Cómo se accede actualmente a estos recursos?
- 6 ¿Cuáles son los problemas si hay múltiples instancias?

Candidatos Típicos:

- Conexión a la base de datos

- Logger del sistema
- Configuración de la aplicación
- Pool de conexiones
- Cache global

Entregable: Lista de 3-5 candidatos a Singleton con justificación.

Actividad 2: Implementar Singleton (20 minutos)

Implementa al menos 2 Singletons en tu sistema.

Ejemplo 1: ConexionBD

```
class ConexionBD {

    private static ConexionBD instancia;
    private Connection conexion;

    private ConexionBD() {
        // Inicializar conexión
        try {
            Class.forName("com.mysql.jdbc.Driver");
            this.conexion = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/logismart",
                "usuario",
                "contraseña"
            );
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    public static ConexionBD obtenerInstancia() {
        if (instancia == null) {
            synchronized (ConexionBD.class) {
                if (instancia == null) {
                    instancia = new ConexionBD();
                }
            }
        }
        return instancia;
    }

    public Connection getConexion() {
```

```

        return conexion;
    }

    public void ejecutarQuery(String sql) {
        try {
            Statement stmt = conexion.createStatement();
            stmt.execute(sql);
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}

```

Ejemplo 2: Logger

```

class Logger {

    private static Logger instancia;
    private PrintWriter writer;

    private Logger() {
        try {
            writer = new PrintWriter(new FileWriter("logismart.log",
true));
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    public static Logger obtenerInstancia() {
        if (instancia == null) {
            instancia = new Logger();
        }
        return instancia;
    }

    public void log(String mensaje) {
        String timestamp = new SimpleDateFormat("yyyy-MM-dd
HH:mm:ss").format(new Date());
        String logMessage = "[" + timestamp + "] " + mensaje;
        writer.println(logMessage);
        writer.flush();
    }
}

```

Preguntas:

- 7 ¿Por qué el constructor es privado?
- 8 ¿Por qué usas double-checked locking?
- 9 ¿Qué pasa si dos hilos llaman a obtenerInstancia() simultáneamente?

Entregable: Código Java de 2-3 Singletons implementados.

Actividad 3: Identificar Candidatos a Factory Method (15 minutos)

Revisa tu diseño y identifica dónde hay múltiples tipos de objetos que necesitan ser creados.

Preguntas a responder:

- 10 ¿Dónde hay múltiples subclases?
- 11 ¿El código cliente conoce todas las subclases?
- 12 ¿Hay condicionales para elegir qué crear?
- 13 ¿Sería beneficioso centralizar la creación?

Candidatos Típicos:

- Tipos de envíos (Express, Standard, Económico)
- Tipos de notificadores (Email, SMS, Push)
- Tipos de vehículos (Auto, Camión, Moto)
- Tipos de usuarios (Cliente, Operador, Administrador)
- Estrategias de asignación de ruta

Entregable: Lista de 3-5 candidatos a Factory Method con justificación.

Actividad 4: Implementar Factory Method (25 minutos)

Implementa al menos 3 Factory Methods en tu sistema.

Ejemplo 1: FabricaDeEnvios

```
interface Envio {  
  
    void enviar();  
    double calcularCosto();  
}
```

```

        int obtenerDiasEntrega();
    }

    class EnvioExpress implements Envio {
        private Cliente cliente;
        private Ubicacion origen;
        private Ubicacion destino;

        public EnvioExpress(Cliente cliente, Ubicacion origen, Ubicacion
destino) {
            this.cliente = cliente;
            this.origen = origen;
            this.destino = destino;
        }

        public void enviar() {
            System.out.println("Enviando por Express...");
        }

        public double calcularCosto() {
            return 100.0; // Costo fijo
        }

        public int obtenerDiasEntrega() {
            return 1; // 24 horas
        }
    }

    class EnvioStandard implements Envio {
        // Similar a EnvioExpress
        public double calcularCosto() { return 50.0; }
        public int obtenerDiasEntrega() { return 5; }
    }

    class EnvioEconomico implements Envio {
        // Similar a EnvioExpress
        public double calcularCosto() { return 25.0; }
        public int obtenerDiasEntrega() { return 10; }
    }

    // Factory
    class FabricaDeEnvios {
        public static Envio crearEnvio(TipoEnvio tipo, Cliente cliente,
Ubicacion origen, Ubicacion destino)
        {
            switch (tipo) {
                case EXPRESS:
                    return new EnvioExpress(cliente, origen, destino);
                case STANDARD:
                    return new EnvioStandard(cliente, origen, destino);
                case ECONOMICO:

```

```

        return new EnvioEconomico(cliente, origen, destino);
    default:
        throw new IllegalArgumentException("Tipo de envío
    inválido");
    }
}

// Uso:
Envio envio = FabricaDeEnvios.crearEnvio(
    TipoEnvio.EXPRESS,
    cliente,
    origen,
    destino
);

```

Ejemplo 2: FabricaDeNotificadores

```

interface Notificador {

    void notificar(String destinatario, String mensaje);
}

class NotificadorEmail implements Notificador {
    public void notificar(String email, String mensaje) {
        System.out.println("Enviando email a " + email + ": " + mensaje);
    }
}

class NotificadorSMS implements Notificador {
    public void notificar(String telefono, String mensaje) {
        System.out.println("Enviando SMS a " + telefono + ": " +
mensaje);
    }
}

class NotificadorPush implements Notificador {
    public void notificar(String usuarioId, String mensaje) {
        System.out.println("Enviando push a " + usuarioId + ": " +
mensaje);
    }
}

// Factory
class FabricaDeNotificadores {
    public static Notificador crearNotificador(TipoNotificador tipo) {
        switch (tipo) {
            case EMAIL:

```

```

        return new NotificadorEmail();
    case SMS:
        return new NotificadorSMS();
    case PUSH:
        return new NotificadorPush();
    default:
        throw new IllegalArgumentException("Tipo de notificador
inválido");
    }
}

}

```

Ejemplo 3: FabricaDeVehiculos

```

interface Vehiculo {

    double obtenerCapacidad();
    double obtenerVelocidadPromedio();
    String obtenerTipo();
}

class Auto implements Vehiculo {
    public double obtenerCapacidad() { return 500.0; } // kg
    public double obtenerVelocidadPromedio() { return 80.0; } // km/h
    public String obtenerTipo() { return "Auto"; }
}

class Camion implements Vehiculo {
    public double obtenerCapacidad() { return 5000.0; }
    public double obtenerVelocidadPromedio() { return 60.0; }
    public String obtenerTipo() { return "Camión"; }
}

class Moto implements Vehiculo {
    public double obtenerCapacidad() { return 100.0; }
    public double obtenerVelocidadPromedio() { return 100.0; }
    public String obtenerTipo() { return "Moto"; }
}

// Factory
class FabricaDeVehiculos {
    public static Vehiculo crearVehiculo(TipoVehiculo tipo) {
        switch (tipo) {
            case AUTO:
                return new Auto();
            case CAMION:
                return new Camion();
            case MOTO:

```

```

        return new Moto();
    default:
        throw new IllegalArgumentException("Tipo de vehículo
    inválido");
    }
}
}

```

Preguntas:

- 14 ¿Por qué el código cliente no conoce las subclases?
- 15 ¿Qué pasa si agregas un nuevo tipo de envío?
- 16 ¿Cómo testearías una Factory?

Entregable: Código Java de 3-4 Factory Methods implementados.

Actividad 5: Integración en LogiSmartController (10 minutos)

Integra los Singletons y Factories en tu LogiSmartController.

```

class LogiSmartController {

    private ConexionBD bd = ConexionBD.obtenerInstancia();
    private Logger logger = Logger.obtenerInstancia();

    public Envio crearEnvio(String tipoEnvio, String idCliente,
                           String origen, String destino) {
        try {
            // Crear envío usando factory
            Cliente cliente = obtenerCliente(idCliente);
            Ubicacion ubicacionOrigen = new Ubicacion(origen);
            Ubicacion ubicacionDestino = new Ubicacion(destino);

            Envio envio = FabricaDeEnvios.crearEnvio(
                TipoEnvio.valueOf(tipoEnvio),
                cliente,
                ubicacionOrigen,
                ubicacionDestino
            );

            // Guardar en BD
            bd.ejecutarQuery("INSERT INTO envios VALUES (...");

```



```

        // Notificar al cliente
        Notificador notificador =
FabricaDeNotificadores.crearNotificador(
        TipoNotificador.EMAIL
    );
    notificador.notificar(cliente.getEmail(), "Envío creado
 exitosadamente");

    // Registrar en log
    logger.log("Envío creado: " + envio.getId());

    return envio;
} catch (Exception e) {
    logger.log("Error al crear envío: " + e.getMessage());
    throw e;
}
}

public boolean asignarRuta(String idEnvio, String tipoVehiculo) {
    try {
        // Crear vehículo usando factory
        Vehiculo vehiculo = FabricaDeVehiculos.crearVehiculo(
            TipoVehiculo.valueOf(tipoVehiculo)
        );

        // Verificar capacidad
        Envio envio = obtenerEnvio(idEnvio);
        if (envio.getPeso() > vehiculo.obtenerCapacidad()) {
            logger.log("Vehículo sin capacidad suficiente");
            return false;
        }

        // Asignar ruta
        bd.ejecutarQuery("UPDATE envios SET vehiculo_id = ... WHERE
id = " + idEnvio);

        logger.log("Ruta asignada: " + idEnvio);
        return true;
    } catch (Exception e) {
        logger.log("Error al asignar ruta: " + e.getMessage());
        return false;
    }
}
}
}

```

Entregable: Código Java del LogiSmartController actualizado con Singletons y Factories.

Actividad 6: Documentación (5 minutos)

Documenta las decisiones de diseño.

Para cada patrón, responde:

- 17 ¿Dónde lo aplicaste?
 - 18 ¿Cuál fue el problema que resolvió?
 - 19 ¿Cómo mejora el diseño?
 - 20 ¿Qué clases o interfaces creaste?
-

3. Entregables

3.1 Documento Markdown

Crea un documento Hito_6_Patrones_Creacionales.md con:

- 21 **Análisis de Candidatos a Singleton**
 - Lista de recursos únicos
 - Justificación de por qué deben ser únicos
- 22 **Implementación de Singletons**
 - Código Java de cada Singleton
 - Explicación de thread safety
 - Ventajas y desventajas
- 23 **Análisis de Candidatos a Factory Method**
 - Lista de tipos de objetos
 - Dónde hay acoplamiento actual
- 24 **Implementación de Factory Methods**
 - Código Java de cada Factory
 - Interfaces y implementaciones
 - Ventajas del desacoplamiento
- 25 **Integración en LogiSmartController**
 - Código del controller actualizado
 - Cómo se usan Singletons y Factories
- 26 **Conclusiones**
 - Cómo cambió el diseño
 - Beneficios de los patrones
 - Próximos pasos

3.2 Código Java

Entrega los siguientes archivos:

- ConexionBD.java - Singleton
- Logger.java - Singleton
- FabricaDeEnvios.java - Factory Method
- FabricaDeNotificadores.java - Factory Method
- FabricaDeVehiculos.java - Factory Method
- LogiSmartController.java - Actualizado

3.3 Diagramas

27 **Diagrama de Clases** - Mostrando Singletons y Factories

28 **Diagrama de Secuencia** - Cómo se crean objetos

4. Criterios de Evaluación

Criterio	Peso	Descripción
Identificación de Singletons	15%	¿Identificaste correctamente qué debe ser único?
Implementación de Singletons	15%	¿Está bien implementado? ¿Thread-safe?
Identificación de Factories	15%	¿Identificaste correctamente dónde hay acoplamiento?
Implementación de Factories	20%	¿Está bien implementado? ¿Desacoplado?
Integración	15%	¿Se integran bien en el controller?
Documentación	20%	¿Se justifican las decisiones? ¿Está bien escrito?

5. Preguntas Guía

Mientras trabajas en el Hito 6:

- 29 **Singleton:** ¿Qué pasa si hay múltiples instancias? ¿Cómo garantizo que haya una única?
 - 30 **Thread Safety:** ¿Qué pasa en un sistema multihilo? ¿Cómo protejo la creación?
 - 31 **Factory:** ¿Dónde hay condicionales que podrían reemplazarse?
 - 32 **Desacoplamiento:** ¿El código cliente conoce las subclases? ¿Debería?
 - 33 **Testabilidad:** ¿Puedo testear fácilmente con mocks?
 - 34 **Escalabilidad:** ¿Es fácil agregar nuevos tipos?
-

6. Recursos

- **Presentación:** [clase_6_presentacion_v1.html](#)
 - **Guión del Orador:** [clase_6_guion_v1.md](#)
 - **Diagramas:** Códigos Mermaid y PlantUML
 - **Documentación:** [Refactoring Guru - Design Patterns](#)
-

7. Notas Importantes

- **Singleton vs Inyección de Dependencias:** En Java moderno, muchos prefieren inyección de dependencias sobre Singleton. Considera ambos enfoques.
 - **Thread Safety:** Asegúrate de que tus Singletons sean thread-safe.
 - **Factory Complexity:** No hagas las factories demasiado complejas. Si crecen mucho, considera Abstract Factory.
 - **Testing:** Piensa en cómo testearías tu código con estos patrones.
-

8. Próximos Pasos

En el Hito 7, veremos:

- **Abstract Factory:** Crear familias de objetos relacionados
- **Builder:** Construir objetos complejos

- **Prototype:** Clonar objetos